

L0 Final Assessment
OOPs Coding / Practical

Employee Name: Nisanth Saravanan
Employee ID: 12576

Question:

Case Study: Personal Task Manager

This C# program simulates a simple to-do list application, demonstrating core object-oriented programming (OOP) principles. The application utilizes a Task struct to represent individual tasks, each containing a description and completion status. An abstract class, TaskPersistence, defines the contract for saving and loading tasks, enabling flexibility in data storage mechanisms. FileTaskPersistence, a concrete implementation, handles file-based persistence. The TodoList class encapsulates the logic for managing a list of tasks, using a vector to store them and employing dependency injection to accept a TaskPersistence object. This allows the TodoList to work with different persistence strategies.

The program features a menu-driven interface,

- 1. Add Task**
- 2. Display Tasks**
- 3. Mark Task Complete**
- 4. Save Tasks**
- 5. Load Tasks**
- 6. Exit**

The code demonstrates abstraction, inheritance, encapsulation, and polymorphism, showcasing a well-structured OOP approach to a basic to-do list application.

Code:

Task.cs

```
struct Task
{
    public int ID;
    public string Description;
    public bool Status;

    public Task(int id, string description)
    {
        this.ID = id;
        this.Description = description;
        this.Status = false;
    }
};
```

Update.cs

//A CONCRETE CLASS DECLARED AND DEFINED WITH THE SOLE PURPOSE OF MARKING TASKS'S STATUS AS TRUE i.e COMPLETED.

```
class Update
{
    public void MarkAsComplete(List<Task> myTasks)
    {
        Console.WriteLine("\nEnter the ID of the task to mark as completed: ");
        int id = int.Parse(Console.ReadLine());

        //CHECKING IF SUCH A TASK IS PRESENT
        bool found = false;
        foreach (Task task in myTasks)
        {
            if (task.ID == id)
            {
                found = true;
                break;
            }
        }

        //IF FOUND:
        if (found)
        {
            Task TaskToComplete = myTasks.Find(task => task.ID == id); //FINDING THE TASK AND
            //PLACING IT IN A TASK VARIABLE.
            myTasks.Remove(TaskToComplete); //DELETES THE SAID TASK, THUS REMOVING IT
            //FROM CURRENT INDEX.
            TaskToComplete.Status = true; //UPDATING STATUS TO TRUE i.e COMPLETED.
            myTasks.Add(TaskToComplete); //RE-ADDING THE TASK TO THE END OF THE LIST.
            Console.WriteLine("The said task has been marked as complete!");
        }
        //IF NOT FOUND:
        else
        {
            Console.WriteLine("The said task is nowhere to be found!");
        }
    }
}
```

TaskPersistence.cs

//A SIMPLE ABSTRACT CLASS THAT DEFINES CONTRACT FOR VARIOUS OPERATIONS SUCH AS ADDING, DISPLAYING, REMOVING AND SEARCHING.

```
abstract class TaskPersistence()
```

```
{
```

```
    public abstract void AddTasks(List<Task> myTasks); //SHOWCASING METHOD OVERRIDING.
```

```
    public abstract void DisplayTasks(List<Task> myTasks); //SHOWCASING METHOD
```

```
OVERLOADING.
```

```
    public abstract void DisplayTasks(List<Task> myTasks, bool status); //SHOWCASING METHOD
```

```
OVERLOADING.
```

```
    public abstract void RemoveTasks(List<Task> myTasks); //SHOWCASING METHOD
```

```
OVERRIDING.
```

```
    public abstract void SearchTasks(List<Task> myTasks); //SHOWCASING METHOD
```

```
OVERRIDING.
```

```
}
```

FileTaskPersistence.cs

//A CONCRETE CLASS THAT IMPLEMENTS AN ABSTRACT CLASS, AND PROVIDES DEFINITION TO THE FUNCTIONS DELARED WITHIN THE ABSTRACT CLASS.

```
class FileTaskPersistence : TaskPersistence
```

```
{
    //SHOWCASING METHOD OVERRIDING.
    public override void AddTasks(List<Task> myTasks)
    {
        Console.WriteLine("\nEnter the ID of the task: ");
        int ID = int.Parse(Console.ReadLine());
        Console.WriteLine("Enter the task description:");
        string description = Console.ReadLine();
```

//Adding the task to the list using the .Add() and a struct constructor.

```
myTasks.Add(new Task(ID, description));
Console.WriteLine($"The new task of ID: {ID}, has been added!");
```

```
}
```

//Showcasing method overriding.

```
public override void DisplayTasks(List<Task> myTasks)
{
    Console.WriteLine("\n=====");
    Console.WriteLine("All tasks:");
    Console.WriteLine("=====");
    foreach (Task task in myTasks)
    {
        Console.WriteLine($"ID: {task.ID}, Description: {task.Description}, Completed?: {task.Status}");
    }
}
```

//Showcasing method overloading.

```
public override void DisplayTasks(List<Task> myTasks, bool status)
{
    Console.WriteLine("\n=====");
    Console.WriteLine("Incomplete tasks:");
    Console.WriteLine("=====");
    foreach (Task task in myTasks)
    {
        if (task.Status == status)
        {
            Console.WriteLine($"ID: {task.ID}, Description: {task.Description}"); //Prints only is the status
        }
    }
}
```

is set to false.

//Showcasing method overriding.

```
public override void RemoveTasks(List<Task> myTasks)
{
```

```
Console.WriteLine("\nEnter the ID of the task to remove: ");
int id = int.Parse(Console.ReadLine());
```

```
//CHECKING IF SUCH A TASK IS PRESENT.
```

```
bool found = false;
foreach (Task task in myTasks)
{
    if (task.ID == id)
    {
        found = true;
        break;
    }
}
```

```
//IF FOUND:
```

```
if (found)
{
```

```
    Task TaskToRemove = myTasks.Find(task => task.ID == id); //FINDING THE TASK AND
    PLACING IT IN A TASK VARIABLE.
```

```
    myTasks.Remove(TaskToRemove); //USING .REMOVE() TO DELETE THE SAID TASK
    FROM ITS INDEX INTO THE ARRAY.
```

```
    Console.WriteLine("THE SAID TASK HAS BEEN REMOVED SUCCESSFULLY!");
```

```
}
```

```
//IF NOT FOUND:
```

```
else
```

```
{
```

```
    Console.WriteLine("THE SAID TASK IS NOWHERE TO BE FOUND!");
```

```
}
```

```
}
```

```
//SHOWCASING METHOD OVERRIDING.
```

```
public override void SearchTasks(List<Task> myTasks)
```

```
{
```

```
    Console.WriteLine("\nEnter the ID of the task to find: ");
```

```
    int id = int.Parse(Console.ReadLine());
```

```
//CHECKING IF SUCH A TASK IS PRESENT.
```

```
bool found = false;
foreach (Task task in myTasks)
{
    if (task.ID == id)
    {
        found = true;
        break;
    }
}
```

```
//IF FOUND:
```

```
if (found)
```

```
{
```

Task TaskToDisplay = myTasks.Find(task => task.ID == id); //FINDING THE TASK AND PLACING IT IN A TASK VARIABLE.

DisplayTask(TaskToDisplay); //FUNCION CALL TO A DISPLAY FUNTION FOR A STANDALONE TASK.

}

//IF NOT FOUND:

else

{

Console.WriteLine("THE SAID TASK IS NOWHERE TO BE FOUND!");

}

}

//SHOWCASING THE USE OF STATIC KEYWORD.

static void DisplayTask(Task task)

{

Console.WriteLine("\n=====");

Console.WriteLine("RESULT:");

Console.WriteLine("=====");

Console.WriteLine(\$"ID: {task.ID}, Description: {task.Description}, Completed?: {task.Status}");

}

}

Program.cs

//AN ENCAPSULATION CLASS THAT WITHHOLDS THE MAIN/CORE LOGIC OF THE TASK MANAGER APPLICATION.

class TodoList

```
{
    public static void Main()
    {
        List<Task> myTasks = new List<Task>();
        FileTaskPersistence fileTaskPersistence = new FileTaskPersistence();
        Update update = new Update();
```

//USING A LABEL-GOTO MECHANISM FOR FLOW CONTROL IN PROGRAM:

checkpoint1:

```
    Console.WriteLine("\n=====");
    Console.WriteLine("TASK MANAGER:");
    Console.WriteLine("=====");
    Console.WriteLine("1. Add Task.");
    Console.WriteLine("2. Display Tasks.");
    Console.WriteLine("3. Display Incomplete Tasks.");
    Console.WriteLine("4. Mark Task Complete.");
    Console.WriteLine("5. Remove Tasks");
    Console.WriteLine("6. Search Tasks");
    Console.WriteLine("0. Exit");
```

```
    Console.Write("\nENTER YOUR CHOICE: ");
    int choice1 = int.Parse(Console.ReadLine());
```

//BASIC SWITCH CASE:

switch (choice1)

```
{
    default:
    {
        Console.WriteLine("\n=====");
        Console.WriteLine("INVALID INPUT, TRY AGAIN!");
        Console.WriteLine("=====");
        goto checkpoint1; //RETURNS TO THE MENU, AWAITING FOR A VALID INPUT.
    }
}
```

case 0:

```
{
    Console.WriteLine("\n=====");
    Console.WriteLine("    LOGGING OUT!");
    Console.WriteLine("=====");
    return; //EXITS PROGRAM
}
```

case 1:

```
{
    fileTaskPersistence.AddTasks(myTasks); //FUNCTION CALL TO ADD A TASK.
```

```

        goto checkpoint1;
    }

    case 2:
    {
        fileTaskPersistence.DisplayTasks(myTasks); //FUNCTION CALL TO DISPLAY ALL
TASKS.
        goto checkpoint1;
    }

    case 3:
    {
        fileTaskPersistence.DisplayTasks(myTasks, false); //FUNCTION CALL TO DISPLAY
INCOMLETE TASKS ONLY.
        goto checkpoint1;
    }

    case 4:
    {
        update.MarkAsComplete(myTasks); //FUNCTION CALL TO MARK A SPECIFIC TASK
AS COMPLETE.
        goto checkpoint1;
    }

    case 5:
    {
        fileTaskPersistence.RemoveTasks(myTasks); //FUNCTION CALL TO REMOVE A
SPECIFIC CLASS.
        goto checkpoint1;
    }

    case 6:
    {
        fileTaskPersistence.SearchTasks(myTasks); //FUNCTION CALL TO SEARCH AND
DISPLA ANY TASK IN PARTICULAR.
        goto checkpoint1;
    }
    }
}
}
}

```


Outputs:

Adding a task:

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 1

ENTER THE ID OF THE TASK: 1
ENTER THE TASK DESCRIPTION:Wash Clothes.
THE NEW TASK OF ID: 1, HAS BEEN ADDED!
```

Displaying all tasks:

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 2

=====
ALL TASKS:
=====

1: Wash Clothes.,    Completed?: False
2: Dry Clothes.,     Completed?: False
```

Marking a task as completed:

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 4

ENTER THE ID OF THE TASK TO MARK AS COMPLETED: 1
THE SAID TASK HAS BEEN MARKED AS COMPLETE!
```

(Verification)

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 2

=====
ALL TASKS:
=====

2: Dry Clothes.,    Completed?: False
1: Wash Clothes.,   Completed?: True
```

Display only incomplete tasks:

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 3

=====
INCOMPLETE TASKS:
=====

2: Dry Clothes.
```

Remove a task:

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 5

ENTER THE ID OF THE TASK TO REMOVE: 1
THE SAID TASK HAS BEEN REMOVED SUCCESSFULLY!
```

(Verification)

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 2

=====
ALL TASKS:
=====

2: Dry Clothes.,    Completed?: False
```

Search for a task:

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 6

ENTER THE ID OF THE TASK TO FIND: 2

=====
RESULT:
=====

2: Dry Clothes.,    Completed?: False
```

Invalid choices:

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 9

=====
INVALID INPUT, TRY AGAIN!
=====
```

Invalid search ID:

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 6

ENTER THE ID OF THE TASK TO FIND: 1
THE SAID TASK IS NOWHERE TO BE FOUND!
```

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit

ENTER YOUR CHOICE: 5

ENTER THE ID OF THE TASK TO REMOVE: 1
THE SAID TASK IS NOWHERE TO BE FOUND!
```

Exit:

```
=====
TASK MANAGER:
=====
1. Add Task.
2. Display Tasks.
3. Display Incomplete Tasks.
4. Mark Task Complete.
5. Remove Tasks
6. Search Tasks
0. Exit
```

```
ENTER YOUR CHOICE: 0
```

```
=====
      LOGGING OUT!
=====
```